Automotive Cause-Effect Chains in the Wild: A Real-World Industrial Benchmark and Generator

Silviu S. Craciunas*†, Matthias Becker‡, Paul Pop*,
*Technical University of Denmark, Kongens Lyngby, Denmark
†NXP Semiconductors, Vienna, Austria
‡KTH Royal Institute of Technology, Stockholm, Sweden
Emails: silviu.craciunas@nxp.com, mabecker@kth.se, paupo@dtu.dk

***Abstract*—Modern automotive systems integrate large numbers of mixed-criticality functions on shared multi-core multi-SoC platforms. Sensor-to-actuator data propagation in these systems is commonly modeled as cause-effect chains that are governed by maximum data-age or reaction-delay budgets. This paper presents the statistical fingerprint of a real-world Advanced Driver-Assistance System (ADAS) use case provided by TrustMotion and introduces a synthetic benchmark comprising 500 task-set instances, organized into five architectural-scale buckets. These instances are derived from the fingerprint by replicating the source compute platform and chain corpus at each scale. We document how the instances were produced, which real-world characteristics they preserve, which aspects they intentionally abstract away, and how researchers can use the data to evaluate mapping, scheduling, and/or end-to-end latency analyzes on inputs whose statistical fingerprint reflects automotive reality.**

I. Introduction

Automotive computing platforms are moving away from hardware-centric designs built around many dedicated and distributed ECUs and toward centralized or zonal software-defined architectures. On these platforms, a growing number of heterogeneous mixed-criticality functions, related to Advanced Driver-Assistance Systems (ADAS), assisted-driving features, and infotainment services, are consolidated onto shared high-performance multi-core multi-SoC hardware [1], [2]. A representative driver-assistance pipeline acquires data from cameras and ranging sensors (ultrasonic, lidar, radar), processes it in sensor-fusion components, and forwards the result to planning, navigation, and control algorithms that ultimately issue actuator commands, e.g., an emergency-brake activation. The correctness of such a pipeline depends not only on each software component meeting its deadline, but also on the temporal consistency of the data as it gets processed through the entire sequence. These end-to-end dependencies are captured as *cause-effect chains*, which are ordered sequences of communicating tasks representing a sensing/processing/actuation pipeline. Cause-effect chains have become a central modelling primitive in automotive timing analysis [3], [4]. A cause-effect chain is a tuple $\zeta = (\langle \tau_1, \tau_2, \ldots, \tau_k \rangle, L, \pi)$ where the task sequence is ordered by data propagation, L is the end-to-end latency budget, and π is a chain-level importance weight. Tasks in a chain may run at different periods and on different cores. Under *reaction delay* the chain bounds the worst-case time from the start of an instance of τ_1, when it reads its input, to the end of the first instance of τ_k that processes this input. Under *data age*, it bounds the age of the data observed at every instance of the chain sink τ_k. Günzel et al. [3] define both semantics and the relationship between them. There have been many works dealing with analyzing and/or enforcing the budgets of cause-effect chains (cf. [4]–[8]).

Progress in the real-time community is sometimes slowed down due to the difficulty of obtaining publicly shareable data sets and problem instances that faithfully reflect industrial reality. Kramer et al. [9] observed that the absence of such benchmarks creates a real risk of divergence between academic research and industrial practice, and proposed a methodology in which aggregate characteristics of real systems are published in place of the raw data so that researchers can validate their solutions on industrially representative input. Concurrent work by Denzinger et al. [10] takes the same fingerprint-style approach on a CARIAD motion-and-drive ECU and ships the *Driverator* framework, where AUTOSAR-style task sets are reconstructed from statistical abstractions of Safety Integrity Levels, basic-software overheads, and memory constraints (Weibull distributions plus per-subject correlation coefficients) while preserving the source IP. We take a similar indirection via statistical-fingerprinting and describe a real automotive use-case specifically detailing the cause-effect-chain level. Furthermore, we also release the generator that can generate based on such fingerprints faithful synthetic use-cases scaled to different sizes within a user-specified bounded deviation (Sec. III-B). We also release synthetic *instances* (500 task-set files, organized in five architectural-scale buckets) generated from that statistical fingerprint using our generator. We hope that in the future we can release more fingerprints of real-world use-cases. Even without the generator, we describe the source use-case in anonymized form so that researchers can produce real-world inspired benchmarks of their own[1].

The remainder of the paper introduces the source use-case together with its task and chain characteristics (Section II), and describes the statistical fingerprint, the synthesis pipeline, and the format and observed properties of the released instances (Section III). We conclude the paper in Section IV.

Version 2.0, 12 October 2026. Revised version of the RTSOPS 2026 paper and the release v2.0 of the benchmark.

[1]The release can be found at https://github.com/dtu-ese

II. The Real-World Use-Case

The benchmark is anchored in a real-world automotive use-case integrated into millions of vehicles which we call the TrustMotion[2] benchmark. Throughout this paper we focus on this strictly computational view. The in-vehicle communication backbone (via, e.g., PCIe or TSN), while industrially important, is outside the scope of the released benchmark. We describe the use-case in three layers (platform anatomy, task model, and chain structure) and conclude with the statistical fingerprint in the spirit of Kramer *et al.* [9], so that researchers can calibrate or refit a generator of their own to obtain faithful synthetic instances.

A. Functional anatomy

The controller integrates three compute processors, each hosting tasks of a distinct functional role:

- **Decision-Making (`DM`).** Trajectory and speed planning. 3 cores.
- **Sensor Fusion (`SF`).** Fuses the upstream sensing streams (radar, lidar, ultrasonic, camera) into the decision-making input. 2 cores.
- **Time-Critical (`TC`).** Front-camera and driver-monitoring image processing. 4 cores.

Although in the original system we use Time-triggered (TT) and Fixed-priority (FP) scheduling, the released benchmark is policy-agnostic and researchers may analyze or schedule the use-case under any policy (FP, EDF, TT) and using any communication semantics (implicit, LET, etc.) as well as partitioned or global approaches. Cause-effect chains in the source either originate at an incoming bus message (typical sources include radar, lidar, ultrasonic, and camera signals) or at an internal sensor-side runnable, and terminate either by emitting an outgoing bus message or by handing data to an internal application task feeding a downstream subsystem. The dataflow direction is roughly: bus or sensor input $\rightarrow$ time-critical or fusion processing $\rightarrow$ decision-making $\rightarrow$ actuator-side bus output or internal handoff.

B. Task model

The source workload contains 102 periodic tasks distributed across the three processors ($62/24/16$ on `DM`/`SF`/`TC`). Each task carries the standard periodic-task parameters: period T, deadline D, worst-case execution time WCET, release offset r (the earliest activation, see below), and an optional jitter constraint J. The jitter J can have different semantics, most commonly it bounds the variation of a task's start time relative to its nominal release across its instances. Other times, like for the TT approach in [2] J_i bounds, for every pair of consecutive instances of task τ_i, the deviation of their start and their finishing times from being exactly one period T_i apart. Every task is bound to a specific CPU (`DM`, `SF`, `TC`) but within the CPU each task is either *pinned* (pre-assigned to a core) or *flex* (the OS or system designer is free to allocate it to any core).

Execution-time information follows the WATERS convention [9]. Every task carries a per-instance worst-case WCET and a per-instance best-case BCET, with the sentinel $\mathrm{BCET} = 0$ reserved for tasks whose source did not record a separate best-case value. The TrustMotion source records only a single execution-time value per task so we set $\mathrm{BCET} = 0$. Fingerprints that include BCET information (such as the WATERS-Bosch fingerprint, built from the published characteristics of Kramer et al. [9]) carry it through unchanged. Three further optional execution-time fields are admitted by the schema and propagated by the generator: (i) $\mathrm{WCET}_{\min}$ lower bound (the lower bound of a per-slot WCET envelope); (ii) an ACET (per-instance average-case time); and (iii) a per-instance execution-time distribution shape (Weibull α, β or empirical distribution). The $\mathrm{WCET}_{\min}$ is present in 8 of the 102 TrustMotion tasks where the communication-stack and per-host middleware runnables have execution time that is driven by the per-slot frame load on the inter-host bus giving a WCET range $[\mathrm{WCET}_{\min}, \mathrm{WCET}]$ reflecting the empirical envelope of the per-slot workload. The underlying parametric form of the load model (frame count, frame size) is a property of the bus, schedule, and the host CPU, both of which the benchmark abstracts away, so we expose only the envelope.

Across the 102 source tasks, some declare a non-zero earliest activation (EA) and/or a non-zero jitter bound. The fingerprint captures EA and jitter at the distribution level rather than per task. For each it records the share of tasks that declare a non-zero value at all $53\,\%$ for EA, $11\,\%$ for jitter. Active EA and J values are binned as a fraction of the task period into a 7-bin histogram, so the distribution shape carries across periods. Note that if the source has many distinct active values spread over a range then binned histograms are created. If the source has only a handful of distinct active values (like in this case the jitter), the fingerprint lists them verbatim with per-100 shares. Hence active jitter values are not binned and the source uses only two discrete ratios, $J/T \in \{0.1, 0.5\}$ (9 and 2 tasks respectively).

C. Cause-effect chains

The use-case contains 39 budget-constrained cause-effect chains. Each chain has an ASIL label, an integer end-to-end latency budget, a priority weight in $[0, 1]$, and an ordered list of runnable references. There are interesting structural patterns shared by some of the chains. On one hand after the receive event there are middleware scheduling-manager and receive-side communication-stack pair (generic plus ASIL-specific) runnables and the symmetric transmit-side comm-stack pair and the middleware again on the send side. The middleware runnable therefore appear *twice* per chain, with 29 of the 39 source chains ($74\,\%$) containing at least one repeated task reference. The comm-stack and middleware tasks invoked on the receive leg are the *same task* invoked on the transmit leg, producing a $\tau_i \prec \ldots \prec \tau_i$ pattern within the chain. Moreover, some in-chain task repetitions

[2]TrustMotion (former TTTech Auto), part of NXP, is a leading software platform product and service provider with a focus on System, Safety, and Security for the Software-Defined Vehicle (4SDV). https://www.trustmotion.com/

occur, encoding the *cyclic stateful pass-through*. Here the chain lists the same runnable at *adjacent* positions $(k, k+1)$, sometimes with multiple successive replicas of the same task. This is some ultrasonic preprocessing or object fusion of the type $\underbrace{\tau_{\mathrm{US}} \to \cdots \to \tau_{\mathrm{US}}}_{n \text{ periods}} \to \underbrace{\tau_{\mathrm{OBF}} \to \tau_{\mathrm{OBF}}}_{m \text{ periods}} \to \cdots$ where τ_{US} is the ultrasonic preprocessing task and τ_{OBF} is the object-fusion main runnable. Both idioms coexist in the same released fingerprint format where the chain is in either case an ordered task sequence with possible repeats and the generator preserves their marginal frequency.

ASIL distribution across the 39 chains is 27 ASIL-B, 4 ASIL-C, 8 unconstrained. No chain carries ASIL-D, although ASIL-D output runnables do appear. 26 of 39 chains begin with a `PDU_IN`, while the remaining start with an internal sensor-side runnable (camera, ultrasonic, etc). 35 of 39 chains end with a `PDU_OUT` and the remaining hand data to an internal application task that feeds a downstream subsystem.

III. Statistical fingerprint

We give the source fingerprint as a set of compact per-dimension tables in the WATERS 2015 style [9], so that a re-implementor can draw periods, WCETs, constraint flags, deadline ratios, chain shapes, and chain budgets independently from the tables that own those axes. All per-task statistics are reported on the 102-task pool.

Table I shows the platform skeleton (CPUs, cores, task counts, cumulative utilization per role). Table II gives the period histogram (102 tasks centred on 10–40 ms, with a 33.3 ms bucket on the `TC` CPU and a single 2.5 ms task on `DM`). Table III gives the per-period WCET range and Table IV the per-CPU pinning prevalence (83 % overall pinning rate). Table V gives the D/T histogram (median 0.83 on the source pool, with 22.5 % implicit-deadline and 11 tasks at $D/T < 0.20$, including the per-core synchronisation tasks). The chain-length distribution is shown in Table VII (multi-modal, with clusters at $L = 7$ and $L \in \{13, 14\}$). The period-transition bigram in Table VIII (with absorbing Start/End states where $115/405$ of all transitions stay at the 5 ms comm-stack inner walk).

Let $T_{\mathrm{th}} = \sum_{j=1}^{n} T_{\tau_j}$ denote the consumer-period sum of the chain over its n scheduled tasks (bus-PDU markers are excluded and the released JSON's chains are task-only, so this matches what the generator computes). Under implicit communication, Davare *et al.* [11] bound the chain's end-to-end latency by $\mathrm{Lat} \le \sum_i (T_{\tau_i} + R_{\tau_i})$. T_{th} is the period-only contribution to that bound, with PDU framing absorbed into the chain's external event boundary. The dimensionless tightness factor $\rho = b/T_{\mathrm{th}}$ records how aggressive the budget is relative to that period-only term. The budget model is shown in Table VI (the empirical ρ histogram and the formula $b = \rho \cdot T_{\mathrm{th}}$). Across the 39 budgeted source chains, ρ has median 0.72, mean 0.99, $\mathrm{p}_{10} = 0.36$, $\mathrm{p}_{90} = 1.43$; 38 % of chains carry $\rho > 1$. A reaction-delay budget can be computed in two ways: as in Sec. I, from the start of the job of τ_1 that reads the input to the end of the first job of τ_k that processes it (the maximum reduced reaction time, MRRT), or from the external event that τ_1 samples (the maximum reaction time, MRT). The MRT additionally contains the sampling delay until τ_1 reads the event. The two differ by between the minimal and the maximal distance of two consecutive starts of τ_1 [3, Thm. 5.3], i.e., by one period T_{τ_1} if τ_1 starts at a fixed offset in its period (e.g., under TT or LET), and by up to $T_{\tau_1} + R_{\tau_1} - C_{\tau_1}$ under fixed-priority scheduling.

TABLE I: Source-side platform skeleton. $|\Gamma|$ is the number of tasks per CPU role. $\sum u$ is computed using $\max C$ for tasks with a recorded compute-time range, giving a per-role upper-bound utilization.

CPU	$\lvert\Gamma\rvert$	$\sum u$ (WCET)
`DM` (3 cores, FP)	62	2.84
`SF` (2 cores, TT)	24	1.81
`TC` (4 cores, TT)	16	3.25
total	102	7.90

TABLE II: Source-side period histogram per CPU role across the 102 tasks. The totals match the $|\Gamma|$ column of Table I. The distribution is centred on 10–40 ms.

T [ms]	`DM`	`SF`	`TC`	total	share
2.5	1	0	0	1	1.0 %
5	10	0	0	10	9.8 %
10	20	7	6	33	32.4 %
20	10	2	3	15	14.7 %
33.3	0	0	5	5	4.9 %
40	13	9	1	23	22.5 %
80	8	6	1	15	14.7 %
total	62	24	16	102	100 %

Together these tables are sufficient to drive a faithful generator, and to evaluate any implementation choice against the source dimension by dimension.

The generator is a direct replicator of the source fingerprint of Section II. For every released instance, all $100k$ tasks and all chains are drawn fresh from the per-dimension distributions in a single anonymized `profile.json`.

A. The fingerprint profile

The fingerprint that drives the generator is captured in a single JSON file (`profile.json`) that contains *only* aggregate statistical distributions. Its top-level keys are:

- `platform` – the per-CPU core counts of the platform to be replicated (no role labels).
- `periods_us` and `per_period` – the period grid and, for every period, the task share (Table II), the WCET data, the WCET envelope used as a clamp at sampling time (Table III), the D/T histogram (Table V), an optional EA histogram (r/T with an explicit "none" mass), the pinning probability, and a chain-visit weight distribution. The WCET data come in one of two forms: a binned *histogram* with adaptive bin widths (used by the TrustMotion fingerprint), or a WATERS-style *aggregate*, i.e., an envelope with optional BCET, ACET, and $\mathrm{WCET}_{\min}$ ranges and a Weibull or

TABLE III: Source-side per-period WCET envelope across the 102 tasks. Each row gives the bucket count and the $[\mathrm{WCET}_{\min}, \mathrm{WCET}_{\max}]$ range observed in the bucket. $\mathrm{WCET}_{\min}$ is the smallest observed WCET in that bucket (over the per-task single WCETs of ordinary tasks and over the per-slot WCET envelopes of the utility-runnable tasks).

T [ms]	n	$\mathrm{WCET}_{\min}$ [µs]	$\mathrm{WCET}_{\max}$ [µs]
2.5	1	100	100
5	10	80	1300
10	33	60	3250
20	15	90	3000
33.3	5	7000	22500
40	23	80	13500
80	15	100	40000

TABLE IV: Source-side per-task constraint prevalence across the 102 tasks. Per-role pinning probability is the per-task fraction with $\mathrm{pinned} = \mathrm{true}$.

constraint	count	share
earliest-activation > 0	54	52.9 %
declared jitter > 0	11	10.8 %
pinned on `DM`	56/62	90.3 %
pinned on `SF`	14/24	58.3 %
pinned on `TC`	15/16	93.8 %
pinned overall	85/102	83.3 %
deadline shorter than period ($D < T$)	79	77.5 %

empirical distribution shape, or, as in the WATERS-Bosch fingerprint, a Weibull ACET distribution with the BCET and WCET factor ranges of Kramer et al. [9].

- `variable_period` (optional) – angle-synchronous or sporadic task classes with a period range (empty for TrustMotion).
- `jitter` – the global jitter marginal: the share of tasks without jitter and the discrete J/T ratios with their shares.
- `chain_count_per_100_tasks` and `same_task_repeat_share` – scalar chain targets (38 and 0.75 for TrustMotion).
- `chains` – four chain-length classes, each with its share, the quartile bands of the chain length (cf. Table VII), and optionally its own period bigram and ρ decile edges. We include also corpus-wide fallbacks for the bigram (Table VIII) and the ρ decile edges (Table VI), the ASIL and chain-priority distributions, and the budget grid `rho_grid_us`. The WATERS-Bosch fingerprint instead uses the chain model of Kramer et al. [9]: 1–3 activation patterns per chain with 2–5 runnables each, and a budget equal to the sum of the patterns' periods.

The profile carries no source identifiers and no verbatim per-task or per-chain values. Every count is reported as a fraction per 100 tasks (or per 100 chains, or per 100 transitions for the chain bigram), pinning probabilities are rounded to a 0.05 granularity, and the per-period WCET histograms use adaptive bin widths. No individual chain's ρ is stored: the ρ pool is reduced from 39 per-chain values to 11 percentile edges $\{p_0, p_{10}, \ldots, p_{100}\}$, from which the generator samples a decile uniformly and then ρ uniformly within it. The released TrustMotion profile is ~ 23 kB.

TABLE V: Source-side deadline-to-period ratio D/T distribution across the 102 tasks. 22.5 % of tasks are implicit-deadline ($D = T$) and the bulk of the tasks sits in $[0.60, 1.00]$.

D/T bin	n	share
$[0.00, 0.20)$	11	10.8 %
$[0.20, 0.40)$	7	6.9 %
$[0.40, 0.60)$	11	10.8 %
$[0.60, 0.75)$	17	16.7 %
$[0.75, 0.90)$	13	12.7 %
$[0.90, 1.00)$	20	19.6 %
$D = T$	23	22.5 %
total	102	100 %

min 0.004, p_{10} 0.150, median 0.825, p_{90} 1.000, max 1.000. Mean 0.704.

TABLE VI: Source-side chain budget model. $T_{\mathrm{th}} = \sum_{j=1}^{n} T_j$ is the consumer-period sum of the chain. The tightness factor $\rho = b/T_{\mathrm{th}}$ is the single-parameter fingerprint stored.

$b = \rho \cdot T_{\mathrm{th}} = \rho \cdot \sum_{j=1}^{n} T_j$, grid = 2.5 ms		
ρ bin	n	share
$[0.20, 0.40)$	4	10.3 %
$[0.40, 0.60)$	4	10.3 %
$[0.60, 0.80)$	12	30.8 %
$[0.80, 1.00)$	3	7.7 %
$[1.00, 1.50)$	12	30.8 %
$[1.50, 2.00)$	0	0.0 %
$[2.00, 3.00)$	4	10.3 %
total	39	100 %

min 0.25, p_{10} 0.36, median 0.72, mean 0.99, p_{90} 1.43, max 2.67 (p_{10}, p_{90}: lower order statistics; the profile stores interpolated decile edges). The 15 chains with $\rho > 1$ are all of length $L \leq 10$; every long chain ($L \geq 13$) has $\rho \leq 0.80$.

B. Generation pipeline

We describe the generation pipeline to generate scaled versions based on the given fingerprint.

a) Platform.: For the platform, each bucket S_k has k replicas of the fingerprint three-CPU platform. We generate $3k$ CPUs with $9k$ cores total (`DM` contributes 3 cores per replica, `SF` 2, `TC` 4). CPU ids are numbered globally 0 to $3k - 1$.

b) Task sampling.: Each instance contains $100k$ tasks drawn independently with the following per-task chain:

1) period T from the period histogram;
2) WCET entry drawn from the source's per-period data, in whichever form the fingerprint provides (histogram or aggregate, see § III-A);
3) the released $\mathrm{WCET}' = \mathrm{round}(\mathrm{WCET} \cdot (1+\delta))$ with $\delta \sim \mathrm{Uniform}(-\varepsilon, +\varepsilon)$, clamped to the per-period envelope. When the fingerprint provides $\mathrm{wcet_min}$ or acet, those are drawn uniformly from their ranges and clamped so that $\mathrm{wcet_min} \leq \mathrm{wcet}$ and $\mathrm{bcet} \leq \mathrm{acet} \leq \mathrm{wcet}$;
4) deadline-ratio r from D/T histogram, $D = \mathrm{round}(r \cdot T)$, clamped to $[C, T]$;
5) a boolean pinned flag sampled from the fingerprint's per-period pinning probability and recorded on the task.

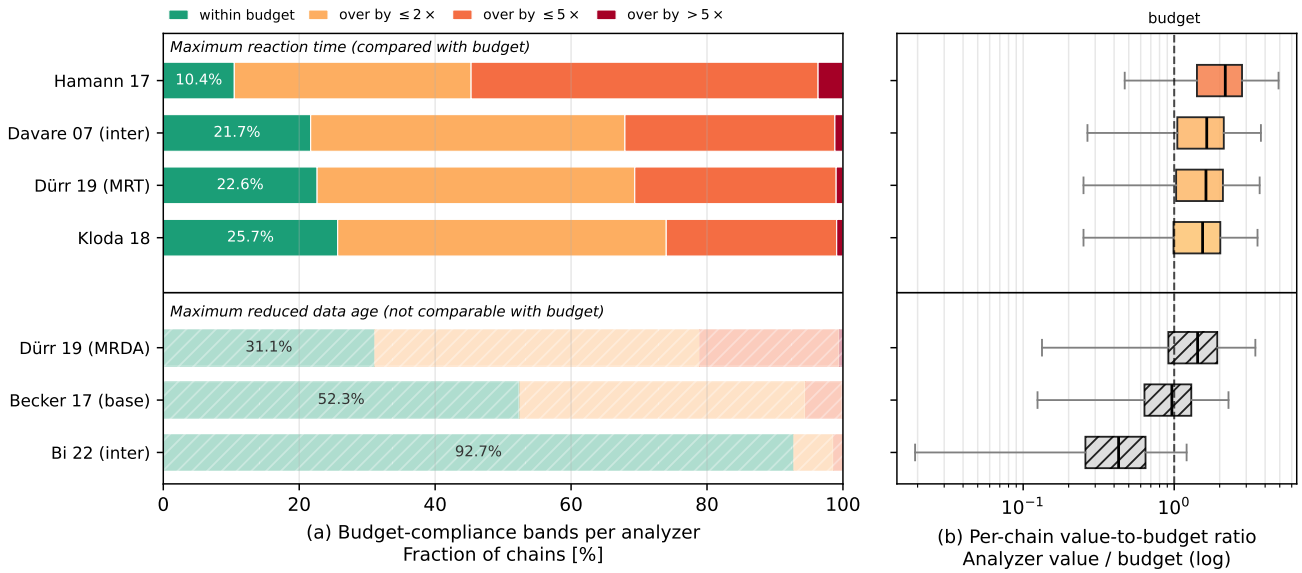


Fig. 1: End-to-end latency values of seven public analyzers on the 100 instances of the `100pct` bucket (3 800 chains) under the reference placement and priorities (partitioned deadline-monotonic fixed-priority scheduling, implicit communication, synchronous periodic releases), each compared with the chain's reaction-delay budget. Hamann, Davare, Dürr (MRT), and Kloda bound the maximum reaction time. Dürr (MRDA) bounds the maximum reduced data age, a different quantity (from sampling to processing) that never exceeds the maximum reaction time, and Becker and Bi compute it per core and sum, which is not a valid bound for multi-core chains. Their rows (pale and hatched, below the line) are not comparable with the budget and are indicative only (Sec. III-C). Panel (a): fraction of chains within budget (green), over by $\leq 2\times$, $\leq 5\times$, and $> 5\times$ (red). Panel (b): per-chain ratio between the analyzer's value and the budget; the dashed line at 1 marks the budget, boxes coloured by the within-budget fraction (whiskers $1.5\times$ IQR, outliers not drawn).

TABLE VII: Source-side chain length histogram (compute tasks only; `PDU_IN`/`PDU_OUT` bus-message markers are not counted, as in the fingerprint).

L	n	share	L	n	share
2	1	2.6 %	9	2	5.1 %
4	3	7.7 %	10	2	5.1 %
5	2	5.1 %	11	2	5.1 %
6	1	2.6 %	12	2	5.1 %
7	8	20.5 %	13	6	15.4 %
8	4	10.3 %	14	4	10.3 %
			15	2	5.1 %

total $= 39$, min $= 2$, mean $= 9.38$, max $= 15$.

After sampling, a capacity pre-check (a greedy packing of all tasks, heaviest first, onto cores with $\sum u \leq 1$) and a 5 % total-capacity safety margin $\sum u_i \leq 0.95 \cdot 9k$ are enforced. Instances that fail are redrawn. The single knob ε is the global `--fingerprint-noise`, default 0.10, and bounds the per-task WCET perturbation as well as the per-chain budget perturbation described below. The reference-schedule post-pass may additionally scale the WCETs of an instance (by a factor α, at least 0.770 in the released data), redraw deadlines, and lower jitter.

Chain end-to-end budgets in the source are not a smooth function of chain length, but they do admit a compact dimensionless fingerprint. At generation time, each synthetic chain draws a ρ uniformly from the source pool and applies a $\pm\varepsilon$ multiplicative perturbation on top:

$$b = \mathrm{round}_{2.5\,\mathrm{ms}}\left(\rho_{\mathrm{src}} \cdot (1+\delta) \cdot T_{\mathrm{th}}^{\mathrm{synth}}\right), \quad \delta \sim \mathrm{Uniform}(-\varepsilon, +\varepsilon) \tag{1}$$

where $T_{\mathrm{th}}^{\mathrm{synth}}$ is the consumer-period sum of the chain's sampled task sequence before the same-task reuse described below. If the resulting b would be smaller than $\sum_i C_i$ along the chain, the ρ draw is retried and the budget ultimately snaps to the next 2.5 ms boundary above the sum-WCET floor so every released chain passes the necessary check $b \geq \sum_i C_i$ (for the MRT, $b \geq T_1 + \sum_i C_i$ is necessary, which 5.4 % of the chains at S_1 and 673 of all 57 000 released chains do not satisfy).

Each instance contains $38k$ chains, that is 38/76/114/152/190 chains at S_1 through S_5. The generator builds every chain in three steps.

First, the chain length is set to $L = k \cdot L_{\mathrm{src}}$, where L_{src} is drawn from the source chain-length histogram in Table VII. Scaling the per-chain length together with the platform replication factor models the realistic case where a $k\times$ larger platform integrates $k\times$ more end-to-end functionality per chain rather than only $k\times$ more independent chains. Callers who prefer pure replication can opt out with `--length-scaling source`, which keeps $L = L_{\mathrm{src}}$ across all buckets.

Second, the period sequence $T_1, \ldots, T_L$ is obtained by walking the chain period bigram in Table VIII from the Start row for up to L steps. The walk terminates earlier when the

TABLE VIII: Source-side chain period-transition bigram with absorbing `Start` (S) and `End` (E) states. Each cell reports the count of from → to transitions between compute tasks (bus-message markers not counted) out of the 405 total transitions across the 39 chains.

from\to [ms]	2.5	5	10	20	33.3	40	80	E
S	0	26	1	3	0	9	0	0
2.5	0	0	0	0	0	0	0	0
5	0	115	15	11	0	0	0	35
10	0	6	44	18	0	22	0	0
20	0	24	5	10	0	1	0	3
33.3	0	0	0	0	0	0	0	0
40	0	5	25	1	0	25	0	1
80	0	0	0	0	0	0	0	0

Rows for the 2.5, 33.3, and 80 ms buckets appear with all-zero transitions: those rates exist in the source workload (Table II) but no chain ever visits a task at any of them. Dividing each cell by its row total gives the transition probability.

TABLE IX: Released synthetic dataset (TrustMotion): per-bucket platform topology and task count. $|\Gamma|$ is the median per-instance task count ($100k$ tasks per replica). $\sum u$ and $\max u/\mathrm{core}$ are computed after the reference-schedule post-pass, $\max u/\mathrm{core}$ under the reference placement. α is the per-instance WCET scaling factor of the reference-schedule post-pass (Sec. III-B).

							α	
Bucket	Files	CPUs	Cores	$\lvert\Gamma\rvert$ (med.)	$\sum u$ (mean)	$\max u/\mathrm{core}$	med.	min
S_1	100	3	9	100	7.60	1.00	1.000	0.770
S_2	100	6	18	200	15.67	1.00	1.000	0.915
S_3	100	9	27	300	24.01	1.00	1.000	0.935
S_4	100	12	36	400	31.96	1.00	0.995	0.935
S_5	100	15	45	500	40.35	1.00	0.990	0.935

TABLE X: Released synthetic cause-effect chain characteristics (TrustMotion). Per-instance chain count is $38k$ (the source's 39 chains per 102 tasks, i.e., 38 per 100 tasks, scaled by k). Under the default `--length-scaling scale`, each chain's length is also multiplied by k (a $k\times$ larger platform integrates $k\times$ more end-to-end functionality per chain). Budgets are drawn from the source's empirical ρ pool (Table VI) with a $\pm\varepsilon$ multiplicative perturbation (Eq. (1)).

		Chain length			Budget [ms]	
Bucket	$\lvert\mathcal{L}_\aleph\rvert$	min	mean	max	min	max
S_1	38	2	8.8	15	2.5	935.0
S_2	76	4	18.5	30	5.0	1200.0
S_3	114	6	27.8	45	10.0	1570.0
S_4	152	8	37.4	60	10.0	2087.5
S_5	190	10	46.8	75	12.5	3005.0

current bigram row offers only an End continuation, which preserves the source's natural termination semantics.

Third, each period T_i is resolved to a concrete task drawn from the instance pool of tasks at period T_i, biased by the per-task visit-weight. Task identities remain unique within the chain. No repeats arise during the walk, which keeps the bigram-driven period sequence and the visit-weight bias intact.

A separate post-pass introduces same-task reuse. For each chain, the generator draws a Bernoulli trial with probability $p =$ `same_task_repeat_share`. On a hit, two distinct positions in $[0, L-1]$ are picked uniformly at random and the task at the first position is copied into the second. The fingerprint target $p = 0.75$ (source: $29/39 \approx 0.744$) is matched within 0.6 percentage points at every scale (74.5–75.1 % of the chains contain a repeated task).

Finally, each chain is assigned an ASIL label sampled from the source's $\{B : 27, C : 4, \mathit{none} : 8\}$ distribution and a budget per Eq. (1).

c) Reference schedule.: Bounding the utilization does not make an instance schedulable. A final, deterministic post-pass therefore makes every instance schedulable under partitioned preemptive fixed-priority scheduling with deadline-monotonic priorities (ties broken by period, then task name). However, schedulability is not checked with respect to the chains, just the task deadlines. Note that in the original use-case, time-triggered scheduling was used on some cores which in general has a higher schedulability bound. The earliest activation is interpreted as the release offset of a task, the jitter as release jitter, and deadlines as relative to the nominal release. The start-jitter and TT-slot semantics of Sec. II-B are not checked. Since the earliest activation and the deadline are sampled independently, $\mathrm{EA} \geq D$ holds for 15 % of the tasks, so EA cannot be read as the start of a window $[\mathrm{EA}, D]$ within the period. In the source, in contrast, D is measured from the

start of the period and $[\mathrm{EA}, D]$ is an execution window, which on the time-triggered CPUs is a slot barely larger than the WCET. As the fingerprint stores EA and D/T independently, the synthetic deadlines are looser than these windows for tasks with $\mathrm{EA} > 0$. Reproducing the windows would require a time-triggered reference schedule. The few offsets that reach the period (15 of 150 000 tasks) are reduced modulo the period. The deadline is enforced to be $D > C$. If a task has $J+C > D$ and cannot be guaranteed to meet its deadline under release jitter, the jitter is lowered to the largest fingerprint ratio $J/T \in \{0.5, 0.1\}$ that fits, or removed. For the schedulability check, tasks are placed in order of decreasing density $(C + J)/D$, each on the least-loaded core (CPUs ordered by mean per-core utilization, then cores by utilization) on which every task still satisfies $R_i \leq D_i$, with the jitter-aware response time $R_i = J_i + w_i$, $w_i = C_i + \sum_{j \in hp(i)} \lceil (w_i + J_j)/T_j \rceil C_j$, where $hp(i)$ is the set of higher-priority tasks on the same core. Because offsets are ignored, the test is sufficient for arbitrary release offsets. If some task fits on no core, all execution times of the instance are scaled by the largest factor α on the grid $\{1, 0.995, 0.990, \ldots\}$ for which the placement succeeds. The resulting placement (CPU and core) is shipped as a reference allocation and α is recorded per instance. Periods, chains and budgets are not modified. Table IX reports α per bucket.

C. Released benchmark

Five scale buckets are released: $S_1, \ldots, S_5$ with $100, 200, 300, 400, 500$ tasks per instance respectively, each bucket containing 100 instances synthesized with different random seeds. Tables IX and X detail some aspects of the generated synthetic cases. Every task ships with its reference placement (CPU and core). The placement is a witness of schedulability under this reference model (by a sufficient analysis), not a constraint: the benchmark remains policy-agnostic, and researchers may re-allocate tasks or apply any other policy. This paper describes version 2.0 of the release. Version 1.0 shipped only a CPU assignment (no core assignment), and with a least-loaded core assignment within those CPUs and deadline-monotonic priorities all of its 500 instances had deadline misses, even without release jitter.

We then ran seven end-to-end analyses of the framework from [12][3] on every chain of the 100-instance `100pct` bucket (3 800 chains) under the reference placement and priorities, implicit communication, and strictly periodic synchronous releases (offsets and jitter set to zero): the Davare et al. bound [11], the maximum-reaction-time (MRT) and maximum-reduced-data-age (MRDA) analyses of Dürr et al. [13], the analysis of Kloda et al. [14], the analysis of Becker et al. without response-time information [4], [5], the analysis of Bi et al. [15], and the bound of Hamann et al. [16]. Davare and Dürr are applied to whole chains. The framework implements the analyses of Kloda and Becker for single-core chains with synchronous periodic releases. We apply them, as well as Hamann's bound, to each per-core segment of a chain and sum, and the framework's interconnected-chain variant of Bi's analysis likewise sums per-core values.

Fig. 1 compares each analyzer's output with the chain's reaction-delay budget, but the outputs are different quantities. Hence, Fig. 1 should not be taken as a reference on how individual analysis methods compare to one another. Davare, Dürr (MRT), and Kloda bound the MRT, i.e., the second way of computing the budget (Sec. III). Summing per-core MRT bounds is sound by the cutting theorem of Günzel et al. [17, Thm. 12]. Hamann's LET bound, which the framework implements as $\sum_i (T_i + D_i)$ [3, Thm. 6.3], is a looser but valid MRT bound here since $R_i \leq D_i$. All four bounds contain the sampling term T_1 once and subtracting it still yields a valid bound for a budget computed from the start of τ_1, for which they are thus conservative by at least T_1. Dürr (MRDA) bounds the maximum reduced data age (the maximum data age of Dürr et al. [17, Sec. VI]), a different quantity, from the sampling of data to its processing, that never exceeds the MRT [3, Thm. 5.3]. Becker and Bi compute it per core and sum, which the cutting theorem does not cover (transferred to the reduced data age, it needs the full data age of all but the last segment [17, Sec. VI]) and which can underestimate it for a multi-core chain. Fig. 1 therefore shows them as a separate, greyed group: their rows show how these analyses' values relate to the budgets, not budget compliance. Chains with repeated tasks required corrections to the framework, e.g. the Becker implementation identified a job's chain position by its task and could return zero or truncated bounds, and the Kloda implementation let a job consume its own output. Both now treat a task repeated at consecutive positions as a later job of that task. The analysis of Dürr et al., stated for distinct consecutive tasks, charges such a step one period, the exact distance between the releases of the two jobs.

D. What the benchmark abstracts away

First, the in-vehicle communication backbone is not modeled and hence cause-effect chains are pure sequences of compute tasks. Second, apart from the reference schedule (Sec. III-B), the schema does not prescribe a scheduling policy. Third, the TrustMotion source workload records only a single per-task WCET, so every released TrustMotion task ships with $\mathrm{bcet} = 0$ (the benchmark's "not measured" sentinel). The schema reserves the `wcet_min` / `acet` / `exec_distribution` optional keys for fingerprints whose source includes those measurements (the second shipped fingerprint, WATERS-Bosch, provides ACETs, from which its BCETs and WCETs follow).

IV. Conclusion

The TrustMotion Cause-Effect Chain Benchmark is derived from a real integrated ADAS controller with 102 tasks on a three-CPU, nine-core platform and 39 ASIL-tagged cause-effect chains, of which 74% contain at least one repeated task reference. We have presented an anonymized statistical fingerprint of this use-case together with a pure-sampling

[3] available at https://github.com/tu-dortmund-ls12-rt/E2EEvaluation

generator, a deterministic post-pass that makes every instance schedulable under a reference fixed-priority schedule, and five scale buckets of synthetic instances (100 instances each, at 100 through 500 tasks per instance). Our hope is that the fingerprint and the generator can be used both by researchers to evaluate their methods and by industry practitioners to release their own use-cases and benchmarks in anonymized form without compromising IP.

ACKNOWLEDGMENTS

The development of the software implementation and experimental (and plotting) infrastructure, as well as the writing of the paper, were assisted by Claude Opus 4.7 (Anthropic). All technical decisions, algorithm design choices, correctness validation, and scientific conclusions were made by the authors.

REFERENCES

[1] G. Niedrist, "Deterministic architecture and middleware for domain control units and simplified integration process applied to ADAS," in *Fahrerassistenzsysteme 2016*. Springer Fachmedien Wiesbaden, 2018.

[2] S. D. McLean, S. S. Craciunas, E. Alexander Juul Hansen, and P. Pop, "Mapping and scheduling automotive applications on adas platforms using metaheuristics," in *Proc. ETFA*, vol. 1, 2020.

[3] M. Günzel, H. Teper, G. v. d. Brüggen, and J.-J. Chen, "End-to-end latency of cause-effect chains: A tutorial," *ACM Trans. Embed. Comput. Syst.*, vol. 24, no. 1, Dec. 2024.

[4] M. Becker, D. Dasari, S. Mubeen, M. Behnam, and T. Nolte, "End-to-end timing analysis of cause-effect chains in automotive embedded systems," *J. Syst. Archit.*, vol. 80, 2017.

[5] ——, "Synthesizing job-level dependencies for automotive multi-rate effect chains," in *Proc. RTCSA*, 2016.

[6] ——, "Mechaniser-a timing analysis and synthesis tool for multi-rate effect chains with job-level dependencies," in *Proc. WATERS*, 2016.

[7] T. Klaus, M. Becker, W. Schröder-Preikschat, and P. Ulbrich, "Constrained data-age with job-level dependencies: How to reconcile tight bounds and overheads," in *Proc. RTAS*, 2021.

[8] M. Günzel and M. Becker, "Optimal task phasing for end-to-end latency in harmonic and semi-harmonic automotive systems," in *Proc. RTAS*, 2025, pp. 164–176.

[9] S. Kramer, D. Ziegenbein, and A. Hamann, "Real world automotive benchmarks for free," in *Proc. WATERS*, 2015.

[10] T. Denzinger, M. Becker, and P. Ulbrich, "Shedding light onto safety integrity level and basic software constraints in a real-world automotive application: Case study with driverator framework," 2026. [Online]. Available: https://arxiv.org/abs/2605.04837

[11] A. Davare, Q. Zhu, M. Di Natale, C. Pinello, S. Kanajan, and A. Sangiovanni-Vincentelli, "Period optimization for hard real-time distributed automotive systems," in *Proc. DAC*, ser. DAC '07. New York, NY, USA: Association for Computing Machinery, 2007, p. 278–283.

[12] R. Edmaier, "Evaluation framework for end-to-end analysis," Master's thesis, TU Dortmund University, 2024. [Online]. Available: https://github.com/tu-dortmund-ls12-rt/E2EEvaluation/blob/main/doc/E2EEvaluation.pdf

[13] M. Dürr, G. V. D. Brüggen, K.-H. Chen, and J.-J. Chen, "End-to-end timing analysis of sporadic cause-effect chains in distributed systems," *ACM Trans. Embed. Comput. Syst.*, vol. 18, no. 5s, Oct. 2019. [Online]. Available: https://doi.org/10.1145/3358181

[14] T. Kloda, A. Bertout, and Y. Sorel, "Latency analysis for data chains of real-time periodic tasks," in *Proc. ETFA*, vol. 1, 2018, pp. 360–367.

[15] R. Bi, X. Liu, J. Ren, P. Wang, H. Lv, and G. Tan, "Efficient maximum data age analysis for cause-effect chains in automotive systems," in *Proc. DAC*. ACM, 2022.

[16] A. Hamann, D. Dasari, S. Kramer, M. Pressler, and F. Wurst, "Communication Centric Design in Complex Automotive Embedded Systems," in *Proc. ECRTS*, vol. 76, Dagstuhl, Germany, 2017.

[17] M. Günzel, K.-H. Chen, N. Ueter, G. v. d. Brüggen, M. Dürr, and J.-J. Chen, "Timing analysis of asynchronized distributed cause-effect chains," in *Proc. RTAS*, 2021, pp. 40–52.